

Lastly

MHacks 2026 build plan for a 4-person team

Track 2: Actually Intelligent (AI). Sponsors: ElevenLabs, Fetch.ai, Neon.

0. Read this first

What we are building. A web app that reads a deceased person's inbox (and bank statements), finds every account they had, shows the family what is still charging, what money is waiting to be claimed and who must be notified, drafts the letters, and uses an ElevenLabs voice agent to phone companies on the family's behalf.

The one demo moment that has to work. A judge watches the dashboard find a \$50,000 life insurance policy the family did not know about, then watches our voice agent call a teammate's phone and cancel the dead person's gym membership live on speaker.

Rules for the whole team.

- Build against the contracts in Section 3. If you need to change a field or endpoint, post it in the team chat before you push.
- Everyone owns specific files (Section 2). Do not edit someone else's file without telling them.
- Push small and often to `main`. Pull before you start each task. Never leave `main` broken.
- The demo runs from the cached result. No live LLM calls during the 2-minute demo except the phone call.
- Cut scope before you cut sleep for everyone. Stretch items are labeled STRETCH. Do them only after the checkpoint they belong to has passed.
- Time is written as H+N, meaning N hours after the team starts this plan. Map it to clock time once you know the submission deadline, and plan to submit at least 1 hour early.

1. Roles

Person	Role	Owns	Delivers
P1	Frontend	<code>static/index.html</code> (and any new static files)	Dashboard, status and assignment UI, coverage checklist, plan-ahead screen, call UI, demo polish
P2	Backend and AI pipeline	<code>pipeline.py</code> , <code>llm.py</code> , <code>server.py</code> , new <code>bank.py</code> , new <code>import_takeout.py</code>	Real LLM extraction with measured recall, active/inactive detection, bank statement detector, Gmail Takeout import, API endpoints
P3	Database and data	new <code>db.py</code> , new <code>schema.sql</code> , <code>generate_inbox.py</code> , <code>data/</code>	Neon Postgres, persistence, Family Hub data model, richer fake data including bank CSV and edge cases
P4	Voice, agents, pitch	<code>calls.py</code> , <code>fetch_agent.py</code> , <code>letters.py</code> , <code>deck</code> , Devpost page, video	Working ElevenLabs phone call, Fetch.ai agent registered, letter quality, pitch, backup video, submission

2. Shared setup (everyone, first 30 minutes)

1. P2 creates a private GitHub repo, unzips `lastly.zip`, commits it, and adds the other three as collaborators.
2. Everyone clones the repo.
3. Create a virtual environment and install dependencies (commands below).
4. Run the offline version end to end and open the dashboard. If this does not work on your laptop, fix it now, not later.
5. Copy `.env.example` to `.env`. Never commit `.env`. Add `.env` and `__pycache__` to `.gitignore` in the first commit.
6. Create a team group chat if you do not have one. Pin this document and the repo link.
7. Each person reads their own section and Section 3 (contracts) before writing code.

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python generate_inbox.py           # writes data/inbox.json and data/truth.json
python pipeline.py --mock          # writes data/estate.json without any API key
uvicorn server:app --reload --port 8000
# open http://localhost:8000 and click "Read Margaret's inbox"
```

Accounts to create now, in parallel: Anthropic API key (P2). Neon account (P3). ElevenLabs account and Twilio trial account (P4). Visit the Fetch.ai, ElevenLabs and AWS sponsor tables in the first two hours to ask about prize rules and credits (P4).

3. Contracts (do not change without telling the team)

3.1 Account object

Every account in the system, whether found in email or bank statements, has this shape. Fields marked NEW are added in this plan.

Field	Type	Meaning
id	string	acct_00, acct_01... stable within one estate
institution	string	Name a family member recognizes, e.g. Planet Fitness
category	enum	subscription, utility, bank, investment, pension, insurance, crypto, payment_app, government, debt, digital_legacy
action	enum	cancel, transfer, claim, notify, memorialize
bucket	enum	leaving, waiting, notify, owed, legacy (derived from category)
amount	number or null	Recurring charge, or latest balance/value
frequency	enum	monthly, annual, balance, one_time, none
next_date	string or null	YYYY-MM-DD
days_until	int or null	Days from today to next_date

Field	Type	Meaning
urgent	bool	Charge or renewal in the next 14 days
why_it_matters	string	One plain sentence for the family
evidence_ids	string[]	Email ids (<code>msg_0123</code>) or bank row ids (<code>bank_017</code>) that prove it
email_count, first_seen, last_seen	int, date, date	Activity range
active (NEW)	bool	False if the last charge is older than 1.5x the billing cycle
sources (NEW)	string[]	["email"], ["bank"] or both
status (NEW)	enum	open, in_progress, done
assigned_to (NEW)	string or null	Family member handling it, e.g. Daniel

3.2 Estate object

GET `/api/estate` returns: `persona` (name, email, age, city, date_of_death, bio), `today`, `stats` (emails, candidates, senders, accounts), `totals` (monthly_drain, charged_since_death, assets_found, debts_found, death_certificates, accounts), and `accounts` (list of account objects, urgent first).

3.3 API endpoints

Method and path	Body	Returns	Owner
POST <code>/api/analyze?mock=&fresh=</code>	none	Estate. Uses cache unless <code>fresh=1</code>	P2
GET <code>/api/estate</code>	none	Estate	P2
GET <code>/api/email/{msg_id}</code>	none	One email (no hidden labels)	P2
GET <code>/api/bank/{row_id}</code> (NEW)	none	One bank statement row	P2
POST <code>/api/letter/{acct_id}</code>	none	{letter}	P4
POST <code>/api/call/{acct_id}</code>	{to_number}	{success, conversation_id, callSid}	P4
GET <code>/api/call/{conversation_id}</code> (NEW, STRETCH)	none	{status, transcript_summary}	P4
POST <code>/api/ask</code>	{question}	{answer, evidence_ids}	P2
PATCH <code>/api/account/{acct_id}</code> (NEW)	{status?, assigned_to?}	Updated account	P3 (DB) + P2 (route)

4. P1: Frontend

Goal: the dashboard tells the story in under 60 seconds without anyone explaining it. Do not redesign. Extend what exists.

Work against mock data until P2 and P3 land their changes. To fake new fields before the backend has them, add them in the browser after fetching the estate (for example default `status` to `open` and `sources` to `["email"]`), then remove the fakes once the API returns them.

Tasks in order

1. **Plan-ahead screen (H+0 to H+1.5).** Before the current start screen, add a screen titled "How Margaret's inbox was connected" showing three options as cards: "She set up Lastly in advance" (selected), "Connect Gmail or Outlook", "Upload a Google Takeout export". Only the selected path needs to work; the other two can show "Coming soon". Button continues to the existing start screen. Purpose: answers the judge question "how do you get access" without words.
2. **Status and assignment (H+1.5 to H+3.5).** On each ledger row add a status chip (Open, In progress, Done) and an assignee label. In the drawer add a status selector and an "Assign to" select with Daniel, Sarah and Me. On change, call `PATCH /api/account/{id}` and update the row. Rows marked Done get a strikethrough name and move to the bottom of their bucket.
3. **Inactive subscriptions (H+3.5 to H+4).** Accounts with `active: false` render greyed out with the label "Stopped charging in 2024" (use `last_seen`). Exclude them from the money-leaving total if P2 has not already.
4. **Sources badges (H+4 to H+5).** Each row shows small text badges for its sources: "Email", "Bank statement". Accounts found only in the bank statement get a short line in the drawer: "No emails. Found only on her Chase statement." Evidence in the drawer must render bank rows as well as emails (`GET /api/bank/{id}` for ids starting with `bank_`).
5. **Coverage checklist (H+5 to H+6.5).** A section under the ledger titled "Where else to look". Checked items: "Email (790 messages)", "Chase bank statement". Unchecked items with one-line instructions: "Credit report: request from the credit bureaus as executor", "Lost life insurance: NAIC Life Insurance Policy Locator", "Unclaimed property: MissingMoney.com and Michigan Unclaimed Property", "Last year's tax return: every 1099 is an account". No links required, plain text is fine.
6. **Call UI (H+6.5 to H+8).** After "Call them for me" succeeds, show a live state: "Calling Planet Fitness..." then "Call in progress" with a pulsing dot. If P4 lands the transcript endpoint, poll it every 3 seconds and show the summary and reference number when the call ends, then set status to Done. If not, show "Call placed" and let the presenter mark it Done manually.
7. **Checkpoint B integration (H+8).** Remove every fake field. Run against the real API with the real pipeline output.
8. **Polish (H+10 to H+13).** Loading and error states for every fetch (no blank screens, no uncaught errors in the console). Check at 1280px and 1440px wide (projector) and at 390px (phone). Make sure the Google Fonts load at the venue. Keyboard: Escape closes the drawer, buttons are focusable.
9. **Demo mode (H+12, 20 minutes).** Add `?demo=1` to the URL to pin the animation timings and skip the plan-ahead screen if the presenter wants. Rehearse the click path with P4.

Done when

- A person who has never seen the app understands what was found and what to do next within one minute.

- Every button in the demo path works against the real API with no console errors.
- Nothing on screen says "mock", "TODO" or shows raw JSON.

5. P2: Backend and AI pipeline

Goal: the real LLM pipeline finds all hidden accounts in Margaret's data, works on a real Gmail export, and merges bank statement findings. You own the API surface; other people add logic behind it.

Tasks in order

1. **Real extraction (H+0 to H+1.5).** Put `ANTHROPIC_API_KEY` in `.env`. Run `python pipeline.py`. It prints `Recall: X/22`. For every miss, read the sender's emails and adjust `EXTRACT_SYSTEM` in `pipeline.py`. Do not hard-code company names. Target 22/22 and all easter eggs found. Commit the resulting `data/estate.json` so everyone has real output.
2. **Chase check (H+1.5).** `no.reply.alerts@chase.com` sends both checking statements and credit card statements. Confirm the pipeline returns two separate accounts for that sender. If it merges them, strengthen the prompt rule about multiple accounts per sender.
3. **Active/inactive (H+1.5 to H+2.5).** In `summarize()`, compute cycle days (monthly 30, annual 365). Set `active = (TODAY - last_seen).days <= 1.5 * cycle` for subscriptions and utilities; `true` for everything else. Exclude inactive accounts from `monthly_drain` and `charged_since_death` and from `urgent`.
4. **Gmail Takeout importer (H+2.5 to H+5).** New file `import_takeout.py`. Input: path to a Takeout `.mbox` file. Output: `data/inbox.json` in the existing format. Details below. Test it on a teammate's own Takeout on that teammate's laptop. Real inbox data never leaves their machine and never goes into the repo.
5. **Bank statement detector (H+5 to H+7).** New file `bank.py` that reads `data/bank.csv` (P3 creates it) and returns account objects with `sources: ["bank"]` and evidence ids `bank_NNN`. Rules below. Call it from `pipeline.run()` after email extraction.
6. **Merge email and bank findings (H+7 to H+8).** In `merge()`, when an email account and a bank account normalize to the same institution, keep one account and union `sources` and `evidence_ids`. Bank descriptors are messy (`PLANET FIT CLUB FEES`), so normalize with a small alias map plus a substring check. If still ambiguous, ask the LLM once with both names: "same company? yes/no".
7. **New endpoints (H+7 to H+8).** `GET /api/bank/{row_id}`. `PATCH /api/account/{acct_id}` that calls P3's `db.update_account()`. Keep the cache rule in `/api/analyze`.
8. **Checkpoint B (H+8).** Full run: `python generate_inbox.py && python pipeline.py`. Recall printed, estate saved to Neon via P3's `db.save_estate()`, dashboard shows sources and active flags.
9. **STRETCH: speed (H+10+).** Use a smaller, faster model for senders the rules mark as obvious (one receipt template repeated) and the main model only for ambiguous senders. Only do this if real Takeout runs are slow.

Takeout importer details

- Use Python's built-in `mailbox.mbox(path)`. Google names the file something like `All mail Including Spam and Trash.mbox`.
- For each message: `From` (parse with `email.utils.parseaddr`), `Subject` (decode with `email.header.decode_header`), `Date` (parse with `email.utils.parsedate_to_datetime`, store `YYYY-MM-DD`), `body` from the first `text/plain` part; if only HTML exists, strip tags with a regex and collapse whitespace. Truncate bodies to 2,000 characters.

- Skip messages from the person's own address and anything in Spam or Trash labels (`X-Gmail-Labels` header).
- Only keep the last 5 years by default (`--years 5`). Life insurance and annual renewals still appear because they email at least yearly.
- Write a `persona` block with the real name blank and `date_of_death` set to today, so the pipeline runs unchanged. Do not write `_truth` fields.
- Print counts: total messages, kept, distinct senders. A 30,000-message inbox should still give a few hundred senders after triage.

Bank detector rules

- Group debit rows by normalized description (uppercase, strip digits, strip `POS`, `ACH`, `DEBIT`, `SQ` *, trailing city/state).
- A group is recurring if it has at least 3 charges, amounts within 5% of the median, and median gap between charges of 25 to 35 days (monthly) or 350 to 380 days (annual).
- Map recurring groups to `category: subscription`, `action: cancel`, `frequency` from the gap, `amount` from the most recent charge, `last_seen` from the most recent date.
- Large regular credits (deposits) with names like `SSA TREAS` or `MPSERS` are income: map to `government` or `pension`, `action: notify`.
- Use the LLM once per group only to turn the cryptic descriptor into a clean institution name.

Done when

- `python pipeline.py` prints 22/22 plus the new accounts P3 adds, with zero manual fixes.
- A teammate's real Takeout produces a sensible list of their real subscriptions.
- Bank-only subscriptions appear on the dashboard with the bank statement as proof.

6. P3: Database and data

Goal: the estate lives in Neon Postgres so the whole family sees the same state (Family Hub), and Margaret's data is rich enough to show every feature in the demo.

Part A: Neon

1. **Create the project (H+0 to H+0.5).** Sign up at neon.com (free plan, no card). Create a project named `lastly`. Copy the connection string into `.env` as `DATABASE_URL`. It must end with `?sslmode=require`. Share it with the team privately, not in the repo.
2. **Install the driver.** Add `psycopg[binary]` to `requirements.txt` and tell the team to reinstall.
3. **Create the schema (H+0.5 to H+1).** Write `schema.sql` (below). Run it from the Neon SQL editor or with `psql "$DATABASE_URL" -f schema.sql`.
4. **Write `db.py` (H+1 to H+3).** Functions: `save_estate(estate) -> estate_id`, `load_estate(estate_id=None)` (latest if none), `update_account(estate_id, acct_id, status=None, assigned_to=None)`, `log_activity(estate_id, acct_id, actor, action)`. If `DATABASE_URL` is not set, every function falls back to the JSON file so offline development keeps working.
5. **Preserve family edits on re-run (H+3).** When the pipeline saves a fresh estate, copy `status` and `assigned_to` from the previous estate for accounts with the same institution and category. Otherwise re-running the pipeline wipes everyone's progress.
6. **Wire into the server (H+3 to H+4, with P2).** `/api/estate` and `/api/analyze` read from `db.load_estate().pipeline.run()` calls `db.save_estate()` at the end. `PATCH /api/account/{id}` calls `db.update_account()` and `db.log_activity()`.
7. **Checkpoint A (H+4).** Two laptops open the dashboard; one marks Netflix Done and assigns Spotify to Sarah; the other refreshes and sees both changes.
8. **STRETCH: activity feed (H+10+).** `GET /api/activity` returns the last 20 rows of activity. P1 shows it as "Sarah cancelled Netflix, 2 hours ago". Only if everything else is done.

```
-- schema.sql
CREATE TABLE IF NOT EXISTS estates (
  id          SERIAL PRIMARY KEY,
  persona     JSONB NOT NULL,
  stats       JSONB NOT NULL,
  totals      JSONB NOT NULL,
  today       DATE  NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE IF NOT EXISTS accounts (
  estate_id   INT REFERENCES estates(id) ON DELETE CASCADE,
  id          TEXT NOT NULL,          -- acct_00
  data        JSONB NOT NULL,         -- full account object from the pipeline
  status      TEXT NOT NULL DEFAULT 'open' CHECK (status IN
('open','in_progress','done')),
  assigned_to TEXT,
  updated_at  TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (estate_id, id)
);

CREATE TABLE IF NOT EXISTS activity (
  id          SERIAL PRIMARY KEY,
  estate_id   INT REFERENCES estates(id) ON DELETE CASCADE,
  account_id  TEXT,
  actor       TEXT,
```

```

    action      TEXT,          -- e.g. status:done, assigned:Sarah
    created_at  TIMESTAMPTZ DEFAULT now()
);

```

Why JSONB: the pipeline's account shape will keep changing tonight. Storing the full object as JSONB means schema changes never block anyone. Only `status` and `assigned_to` are real columns because they are edited by people.

Part B: Data (start in parallel with Part A, finish by H+6)

Edit `generate_inbox.py`. Every new item must also go into `truth.json` so P2's recall check stays honest. Each item below exists to demonstrate one feature.

Add this	How	Demonstrates
Apple bundled receipt	Monthly email from Apple listing three line items: iCloud+ \$2.99, Calm \$14.99, Paramount+ \$7.99. Remove the standalone iCloud account.	LLM reads line items, not just senders
Stopped subscription	Hulu receipts monthly from Jan 2024 to Jun 2024, then nothing	Active/inactive detection
Account with no money mentioned	One "Welcome to Delta SkyMiles" email from 2019 plus two "Your miles statement" emails	Finding accounts from welcome emails
Free trial	"Your free trial of Peacock ends on 2026-09-20"	Trial converting to a paid charge after death
Bank-only subscription	No emails at all. Appears only in <code>bank.csv</code> as <code>SQ *YPSI YOGA COLLECTIVE \$45.00</code> monthly	Why multiple sources matter
Messier noise	40 more noise emails that contain trigger words but are not accounts: "Your Kohl's account has rewards", "Statement from your church giving", phishing "Your account is suspended"	LLM rejects false positives

Bank statement file. Write `data/bank.csv` from the same generator: Chase checking, Jan 2025 to the date of death, columns `id, date, description, amount` (negative for debits). Include matching charges for every email subscription (with cryptic descriptors like `NETFLIX.COM 866-579-7172 CA, PLANET FIT CLUB FEES`), the yoga charge with no email, the MetLife monthly premium (`METLIFE PREM`), deposits for Social Security (`SSA TREAS 310`) and MPSERS, and 150 random one-off purchases (groceries, gas, pharmacy) as noise.

Done when

- Changes on one laptop show up on another after refresh.
- Re-running the pipeline does not erase statuses or assignments.
- Every item in the table above appears in the dashboard and in `truth.json`.

7. P4: Voice, agents and pitch

Goal: the phone call works reliably on stage, the Fetch.ai agent is registered and answers questions, and the pitch, Devpost page and backup video are finished early.

Part A: ElevenLabs phone call (highest risk, start first)

1. **Accounts (H+0 to H+0.5).** Create an ElevenLabs account and a Twilio trial account. Ask at the ElevenLabs table about hackathon credits. Check how many conversation minutes your plan includes; each test call uses some.
2. **Twilio number (H+0.5 to H+1).** Get a free trial phone number in Twilio. Under Verified Caller IDs, verify every teammate's phone number. Trial accounts can only call verified numbers.
3. **Create the agent (H+1 to H+1.5).** In ElevenLabs, create a conversational agent. Paste `AGENT_PROMPT` from `calls.py` as the system prompt and `FIRST_MESSAGE` as the first message. Pick a calm, clear voice. Make sure the `{{variables}}` in the prompt are registered as dynamic variables.
4. **Connect the number (H+1.5 to H+2).** In ElevenLabs phone numbers, import the Twilio number with your Twilio Account SID and Auth Token, and assign the agent to it. Put `ELEVENLABS_API_KEY`, `ELEVENLABS_AGENT_ID` and `ELEVENLABS_PHONE_NUMBER_ID` in `.env`.
5. **First call (H+2 to H+3).** With the server running, place a call from the terminal (command below). The phone should ring within seconds. Fix anything wrong now.
6. **Twilio trial notice.** Trial calls play a short Twilio message first and the person answering must press a key before the agent speaks. Rehearse that. If it looks bad on stage, upgrading the Twilio account with a small payment removes it; decide as a team.
7. **Role-play script (H+3 to H+4).** One teammate plays "Planet Fitness member services". Write their lines: greet, ask for the member's name, ask for the date of death, say they need a death certificate emailed to an address, give reference number PF-20931, confirm cancellation. Run it 5 times. The agent must repeat the reference number back.
8. **Checkpoint A (H+4).** Three successful calls in a row from the dashboard button, not just the terminal.
9. **STRETCH: transcript (H+10+).** After the call, fetch the conversation from the ElevenLabs API by `conversation_id`, have the LLM extract `{cancelled: bool, reference_number, next_steps}`, return it from `GET /api/call/{conversation_id}`, and let P1 mark the account Done automatically.

```
curl -X POST localhost:8000/api/call/acct_XX \  
-H "content-type: application/json" \  
-d '{"to_number": "+1734XXXXXXX"}' \  
# acct_XX = Planet Fitness id from GET /api/estate
```

Non-negotiable: the agent says it is an AI calling on behalf of the family. It is already in the prompt. Do not remove it. Judges will ask about it, and it is the right thing to do.

Part B: Fetch.ai agent

1. **Rules (H+0 to H+2).** At the Fetch.ai table, ask exactly what this year's prize requires (Agentverse registration, Chat Protocol, ASI:One, anything else). Adjust the plan to match.
2. **Run it (H+4 to H+5).** Set `AGENT_SEED` in `.env` to a long random phrase and keep it fixed. Run `python fetch_agent.py` with the server running. Open the Agent Inspector link it prints and connect via Mailbox to register on Agentverse.
3. **Test (H+5 to H+6).** Ask through Agentverse or ASI:One: "Did Margaret have life insurance?" The reply must mention MetLife and cite an email id.

4. **Profile.** Give the agent a clear name, description and a few example questions on Agentverse so judges can find and test it.

Part C: Letters

With the real LLM on, generate a letter for one account in every action type (cancel, transfer, claim, notify, memorialize). Check: correct name and date of death, no invented account numbers, under 180 words, clear request, placeholders for executor contact details. Fix problems in the `SYSTEM` prompt in `letters.py`.

Part D: Pitch, Devpost, video

1. **Deck (H+6 to H+9), 5 slides max.** 1) The problem: families spend over a year settling affairs and miss accounts nobody knew about. 2) Live demo (no slide content, switch to the app). 3) How it works: inbox and bank statements, LLM extraction, voice agent. 4) Accuracy: recall number from P2 and what the coverage checklist covers. 5) Sponsors used and what is next.
2. **Devpost (H+9 to H+11).** Name, one-line tagline, inspiration, what it does, how we built it (name every sponsor tool), challenges, accomplishments, what's next. Add screenshots and the video. Save as draft early.
3. **Backup video (H+13 to H+14).** Screen-record the full demo including a phone call with the audio. Under 2 minutes. Upload it to Devpost and keep a local copy on two laptops.
4. **Rehearse (H+14 to H+15).** Full demo at least 5 times with the exact laptop, browser and phone you will use.

8. Timeline and checkpoints

Time	P1 Frontend	P2 Backend	P3 Database and data	P4 Voice and pitch
H+0 to 1	Setup, plan-ahead screen	Setup, real pipeline run	Neon project and schema	Accounts, Twilio number, sponsor tables
H+1 to 4	Status and assignment UI, inactive styling	Recall fixes, active/inactive, start Takeout importer	<code>db.py</code> , wire server, data additions start	Agent setup, first call, role-play script
H+4 Checkpoint A	UI works on mock with new fields	Real <code>estate.json</code> committed, recall printed	Two laptops share state via Neon	3 calls in a row from the button
H+4 to 8	Sources badges, coverage checklist, call UI	Takeout importer, bank detector, merge, endpoints	Finish data table, <code>bank.csv</code> , truth updates	Fetch.ai agent, letters QA, deck draft
H+8 Checkpoint B	Everything on real API	Full pipeline incl. bank on <code>main</code>	Edits survive pipeline re-run	Agent answers on Agentverse
H+8 to 10	Sleep shift 1 (P1, P3)	Bug fixes	Sleep shift 1	Devpost draft

Time	P1 Frontend	P2 Backend	P3 Database and data	P4 Voice and pitch
H+10 to 12	Polish, demo mode	Sleep shift 2 (P2, P4)	Activity feed STRETCH	Sleep shift 2
H+12 to 13	Final polish	Real Takeout demo on a teammate	Final data check	Transcript STRETCH
H+13 Feature freeze	Bug fixes only from here	Bug fixes only	Bug fixes only	Bug fixes only
H+13 to 15	Rehearse	Final cached run, commit it	Back up DB (export JSON)	Video, Devpost, rehearse
Deadline minus 1 hour	Submit	Submit	Submit	Submit on Devpost

At each checkpoint: stop for 10 minutes, everyone pulls `main`, one person runs the full demo path on the presenting laptop, and the team agrees on what to cut if behind. If a checkpoint is missed by more than an hour, drop that person's STRETCH items and the lowest-value task in their list, in that order.

9. Demo script (2 minutes)

1. **Open (15s).** "Raise your hand if you know every subscription your parents pay for." Pause. "When someone dies, their family has to find every account, cancel every charge and claim every asset, mostly from memory. It takes over a year."
2. **Plan-ahead screen (10s).** "Margaret set up Lastly before she died. Her son Daniel is the executor."
3. **Read the inbox (20s).** Click. Counters run. Read the sentence on screen aloud.
4. **Urgent alert (10s).** "Amazon Prime renews in 3 days. And about \$400 has already been charged since she died."
5. **The surprise (20s).** Scroll to MetLife. "Her family didn't know this existed. A \$50,000 policy. Here's the email that proves it." Open the drawer.
6. **The call (35s).** Open Planet Fitness. "She hadn't been to the gym since 2023. Daniel doesn't want to make 20 phone calls." Click Call. Teammate's phone rings on speaker. Let the agent run until the reference number.
7. **Close (10s).** "Lastly finds what you didn't know was there, and makes the calls for you."

10. Judge questions and our answers

Question	Answer
How do you get into a dead person's email?	Usually the family already has the unlocked phone or laptop. Otherwise Apple Legacy Contact, Google Inactive Account Manager, or the person sets up Lastly in advance. We never need their passwords for other accounts: companies close accounts through their deceased-customer process with a death certificate.

Question	Answer
Isn't this a privacy problem?	It's built for the executor, who has legal authority over the estate. Nothing is sent and no call is placed without the family approving it, and the agent always says it's an AI.
Doesn't Empathy already do this?	Empathy guides families through tasks with checklists and care managers. We focus on discovery: finding accounts nobody knew about, from the actual data, with proof, and then acting on them.
Doesn't Apple Legacy Contact do this?	Apple gives you access to the raw data and explicitly excludes subscriptions and payment info. We turn that data into a list of accounts and actions. Apple is one of our inputs.
How accurate is it?	On our test estate it found X of Y hidden accounts (use P2's number). And we don't claim to find everything: the coverage checklist tells the family exactly where else to look.
What if the LLM hallucinates an account?	Every account links to the email or bank row that proves it. No proof, no account.
Is this legal advice?	No. It finds accounts and drafts letters for the family to review. We tell users to check with a probate lawyer for legal questions.

11. Risks

Risk	Mitigation	Owner
Phone call fails on stage (signal, Twilio, credits)	Backup video with audio. Test on venue Wi-Fi and cell signal at H+12. Keep credits topped up.	P4
LLM slow or down during demo	Demo runs from cached estate. Only the call is live.	P2
Neon unreachable	<code>db.py</code> falls back to JSON. Keep a JSON export of the final estate.	P3
Real Takeout leaks private data	Run only on the owner's laptop. Never commit it. Never show it on the projector without the owner's OK.	P2
Merge conflicts late at night	File ownership in Section 1. Pull before every task. Feature freeze at H+13.	All
Team burnout	Two sleep shifts. Nobody works the full night.	All

12. Submission checklist

- Devpost submitted with all four team members added, at least 1 hour before the deadline.
- Correct track selected (Actually Intelligent) and every sponsor prize we qualify for selected.
- Repo link works and the README run instructions are correct. `.env` is not in the repo.
- Backup video uploaded and playable.

- Presenting laptop charged, phone charged, speakerphone tested, browser zoom set, notifications off.
- Cached `estate.json` and Neon data are the final versions. Statuses reset to Open before judging.